

I.A. Data type of all columns in the “customers” table.

☐	Field name	Type	Mode	Key	Collation	Default Value	Policy Tags ⓘ	Data Policies	Description
☐	customer_id	STRING	NULLABLE	-	-	-	-	-	-
☐	customer_unique_id	STRING	NULLABLE	-	-	-	-	-	-
☐	customer_zip_code_prefix	INTEGER	NULLABLE	-	-	-	-	-	-
☐	customer_city	STRING	NULLABLE	-	-	-	-	-	-
☐	customer_state	STRING	NULLABLE	-	-	-	-	-	-

Insight: Most of the fields use **STRING** (4 fields) and one of the field date type is **INTEGER**.

I.B. Get the time range between which the orders were placed.

Job information		Results	Visualization	JSON	E
Row	first_order ▼	last_order ▼			
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC			

Query:

```
select min(order_purchase_timestamp) as first_order,
       max(order_purchase_timestamp) as last_order
from `BusinessCaseTargetSQL.orders`;
```

Insight:The actual time range during which orders were placed is from **2016-09-04 21:15:19 UTC** to **2018-10-17 17:30:18 UTC**.

I.C. Count the Cities & States of customers who ordered during the given period.

Job information		Results	Visualization
Row	Cities ▼	States ▼	
1	179	21	

```
select count(distinct c.customer_city) as Cities ,count(distinct c.customer_state) as
States
from `BusinessCaseTargetSQL.customers` as c join `BusinessCaseTargetSQL.orders` as o
on c.customer_id= o.customer_id
where extract(YEAR from o.order_purchase_timestamp)= 2016;
```

Insight: In 2016, customers placed orders from 179 cities across 21 different states.

II.A. Is there a growing trend in the no. of orders placed over the past years?

Job information		Results	Visualizat
Row	year ▼	Total_orders ▼	
1	2016	329	
2	2017	45101	
3	2018	54011	

Query:

```
SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
    COUNT(order_id) AS Total_orders
FROM `BusinessCaseTargetSQL.orders`
GROUP BY EXTRACT(YEAR FROM order_purchase_timestamp)
ORDER BY year ASC
```

Insight:

- There was a massive increase in orders from 2016 to 2017, indicating a period of rapid growth or expansion of the platform.
- The growth continued from 2017 to 2018, showing that the business's positive momentum was sustained.

II.B. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Job information		Results	Visualizatio
Row	month ▼	no_of_orders ▼	
1	1	8069	
2	2	8508	
3	3	9893	
4	4	9343	
5	5	10573	
6	6	9412	
7	7	10318	
8	8	10843	
9	9	4305	
10	10	4959	
11	11	7544	
12	12	5674	

```
Query :SELECT
    EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
    COUNT(order_id) AS no_of_orders
FROM
    `BusinessCaseTargetSQL.orders`
GROUP BY
    month
ORDER BY
    month ASC;
```

Insight:

The order volume is consistently high during the first half of the year, but shows a significant dip in September. The data then indicates a strong rebound in November.

II.C. During what time of the day, do the Brazilian customers mostly place their orders?
(Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

Job information		Results	Visualization	JS
Row	time_of_day	total_orders		
1	Afternoon	38135		
2	Night	28331		
3	Mornings	27733		
4	Dawn	5242		

```
Query: SELECT
    CASE
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN
            'Dawn'
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN
            'Mornings'
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN
            'Afternoon'
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23 THEN
            'Night'
        END AS time_of_day,
    COUNT(order_id) AS total_orders
FROM
    `BusinessCaseTargetSQL.orders`
GROUP BY
    time_of_day
ORDER BY
    total_orders DESC;
```

Insight:

Afternoon has the largest share of total orders. The **Night** and **Morning** hours also have a significant volume of orders, while the early morning **Dawn** period has the lowest.

III.A. Get the month on month no. of orders placed in each state.

Job information		Results	Visualization	JSON	Execution details	Ex
Row	customer_state	order_month	total_orders			
1	AC	2017-01	2			
2	AC	2017-02	3			
3	AC	2017-03	2			
4	AC	2017-04	5			
5	AC	2017-05	8			
6	AC	2017-06	4			
7	AC	2017-07	5			
8	AC	2017-08	4			
9	AC	2017-09	5			
10	AC	2017-10	6			
11	AC	2017-11	5			
12	AC	2017-12	5			

Query: **SELECT**

```
    c.customer_state,
    FORMAT_TIMESTAMP('%Y-%m', o.order_purchase_timestamp) AS order_month,
    COUNT(o.order_id) AS total_orders
FROM
    `BusinessCaseTargetSQL.orders` AS o
JOIN
    `BusinessCaseTargetSQL.customers` AS c
ON
    o.customer_id = c.customer_id
GROUP BY
    c.customer_state, order_month
ORDER BY
    c.customer_state ASC, order_month ASC;
```

Insight:

Across all months, customers placed a total of 99,441 orders. SP emerged as the leading state, contributing 41,746 orders.

III.B. How are the customers distributed across all the states?

Job information		Results	Visualization	JSON
Row	customer_state ▼	NO_OF_CUST ▼		
1	SP	41746		
2	RJ	12852		
3	MG	11635		
4	RS	5466		
5	PR	5045		
6	SC	3637		
7	BA	3380		
8	DF	2140		
9	ES	2033		
10	GO	2020		

Query: `SELECT`

```
    customer_state,
    count(distinct customer_id) as NO_OF_CUST
FROM
    `BusinessCaseTargetSQL.customers`
group by customer_state
order by NO_OF_CUST desc;
```

Insight:

SP has the highest number of unique customers at 41,746. States such as RR and AP have significantly fewer customers.

IV.A. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

Row	cost_2017	cost_2018	pct_increase
1	3669022.12	8694733.84	136.98

Query: `SELECT`

```
ROUND(MAX(CASE WHEN order_year = 2017 THEN total_cost END),2) AS cost_2017,
ROUND(MAX(CASE WHEN order_year = 2018 THEN total_cost END),2) AS cost_2018,
ROUND(((MAX(CASE WHEN order_year = 2018 THEN total_cost END) -
        MAX(CASE WHEN order_year = 2017 THEN total_cost END))
        / MAX(CASE WHEN order_year = 2017 THEN total_cost END) * 100), 2)
AS pct_increase
FROM (
    SELECT
        EXTRACT(YEAR FROM o.order_purchase_timestamp) AS order_year,
        SUM(p.payment_value) AS total_cost
    FROM `BusinessCaseTargetSQL.orders` o
    JOIN `BusinessCaseTargetSQL.payments` p
        ON o.order_id = p.order_id
    WHERE EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8
    GROUP BY order_year
) t;
```

Insight:

From Jan–Aug 2017 to the same period in 2018, the total order cost grew significantly by 136.98%.

IV.B. Calculate the Total & Average value of order price for each state.

Job information	Results	Visualization	JSON	Execution de
Row	customer_state ▼	total_order_value ▼	avg_order_value ▼	
1	SP	5998226.96	137.5	
2	RJ	2144379.69	158.53	
3	MG	1872257.26	154.71	
4	RS	890898.54	157.18	
5	PR	811156.38	154.15	
6	SC	623086.43	165.98	
7	BA	616645.82	170.82	
8	DF	355141.08	161.13	
9	GO	350092.31	165.76	
10	ES	325967.55	154.71	
11	PE	324850.44	187.99	

Query: [SELECT](#)

```
c.customer_state,
Round(SUM(p.payment_value),2) AS total_order_value,
ROUND(AVG(p.payment_value), 2) AS avg_order_value
FROM `BusinessCaseTargetSQL.customers` c
JOIN `BusinessCaseTargetSQL.orders` o
  ON c.customer_id = o.customer_id
JOIN `BusinessCaseTargetSQL.payments` p
  ON o.order_id = p.order_id
GROUP BY c.customer_state
ORDER BY total_order_value DESC;
```

Insight:

SP leads with the highest total order value ,which is much greater than RJ and MG. Smaller states like PB, AL, and AC show some of the highest average order values despite low overall contribution.

IV.C. Calculate the Total & Average value of order freight for each state.

Row	customer_state	total_freight_value	avg_freight_value
1	SP	718723.07	15.15
2	RJ	305589.31	20.96
3	MG	270853.46	20.63
4	RS	135522.74	21.74
5	PR	117851.68	20.53
6	BA	100156.68	26.36
7	SC	89660.26	21.47
8	PE	59449.66	32.92
9	GO	53114.98	22.77
10	DF	50625.5	21.04
11	ES	49764.6	22.06
12	CE	48351.59	32.71

Query: `SELECT`

```
    c.customer_state,
    ROUND(SUM(oi.freight_value), 2) AS total_freight_value,
    ROUND(AVG(oi.freight_value), 2) AS avg_freight_value
FROM `BusinessCaseTargetSQL.customers` c
JOIN `BusinessCaseTargetSQL.orders` o
    ON c.customer_id = o.customer_id
JOIN `BusinessCaseTargetSQL.order_items` oi
    ON o.order_id = oi.order_id
GROUP BY c.customer_state
ORDER BY total_freight_value DESC;
```

Insight:

SP contributes the highest freight value and PB, RR, and RO bear the highest average freight costs.

V.A. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- **time_to_deliver** = order_delivered_customer_date - order_purchase_timestamp.
- **diff_estimated_delivery** = order_estimated_delivery_date - order_delivered_customer_date

Row	order_id	time_to_deliver	diff_estimated_d...
1	65d1e226dfaeb8cdc42f665422...	35	16
2	2c45c33d2f9cb8ff8b1c86cc28...	30	28
3	1950d777989f6a877539f53795...	30	-12
4	bfb0f9bdef84302105ad712db...	54	-36
5	98974b076b01553d49ee64679...	43	6
6	c4b41c36dd589e901f6879f25a...	36	14
7	d2292ff2201e74c5db154d1b7a...	29	20
8	95e01270fcb9e986342340010...	30	19
9	ed8c7b1b3eb256c70ce0c7423...	44	5
10	5cc475c7c03290048eb2e742c...	68	-18

Query: **SELECT**

```
    order_id,  
    DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS  
time_to_deliver,  
    DATE_DIFF(order_estimated_delivery_date, order_delivered_customer_date,  
DAY) AS diff_estimated_delivery  
FROM `BusinessCaseTargetSQL.orders`  
WHERE order_delivered_customer_date IS NOT NULL  
      AND order_purchase_timestamp IS NOT NULL  
      AND order_estimated_delivery_date IS NOT NULL;
```

Insight : Out of 96476 orders 6535 orders was delivered late

V.B. Find out the top 5 states with the highest & lowest average freight value.

Row	customer_state	avg_freight_value
1	SP	15.15
2	PR	20.53
3	MG	20.63
4	RJ	20.96
5	DF	21.04
6	PI	39.15
7	AC	40.07
8	RO	41.07
9	PB	42.72
10	RR	42.98

```
Query: SELECT *
      FROM (
        SELECT *
        FROM (SELECT
              c.customer_state,
              ROUND(AVG(oi.freight_value), 2) AS avg_freight_value
            FROM `BusinessCaseTargetSQL.customers` c
            JOIN `BusinessCaseTargetSQL.orders` o
              ON c.customer_id = o.customer_id
            JOIN `BusinessCaseTargetSQL.order_items` oi
              ON o.order_id = oi.order_id
            GROUP BY c.customer_state
            ORDER BY avg_freight_value DESC
            LIMIT 5) union all
        (
          SELECT
            c.customer_state,
            ROUND(AVG(oi.freight_value), 2) AS avg_freight_value
          FROM `BusinessCaseTargetSQL.customers` c
          JOIN `BusinessCaseTargetSQL.orders` o
            ON c.customer_id = o.customer_id
          JOIN `BusinessCaseTargetSQL.order_items` oi
            ON o.order_id = oi.order_id
          GROUP BY c.customer_state
          ORDER BY avg_freight_value ASC
```

```

LIMIT 5 )
)
ORDER BY avg_freight_value ASC;

```

Insight : Freight costs are highest in RR, PB, RO, AC, and PI because these states are far away and harder to reach. On the other hand, SP has the lowest freight costs, and PR, MG, RJ, and DF are also cheaper, probably because they have better transport systems.

V.C. Find out the top 5 states with the highest & lowest average delivery time.

Row	customer_state	avg_delivery_time
1	SP	8.3
2	PR	11.53
3	MG	11.54
4	DF	12.51
5	SC	14.48
6	PA	23.32
7	AL	24.04
8	AM	25.99
9	AP	26.73
10	RR	28.98

Query :

```

SELECT *
FROM (
  SELECT *
  FROM (
    SELECT
      c.customer_state,
      ROUND(AVG(DATE_DIFF(o.order_delivered_customer_date,
o.order_purchase_timestamp, DAY)), 2) AS avg_delivery_time
    FROM BusinessCaseTargetSQL.customers c
    JOIN BusinessCaseTargetSQL.orders o
      ON c.customer_id = o.customer_id
    WHERE o.order_delivered_customer_date IS NOT NULL

```

```

        GROUP BY c.customer_state
        ORDER BY avg_delivery_time DESC
        LIMIT 5
    )
    UNION ALL
    SELECT *
    FROM (
        SELECT
            c.customer_state,
            ROUND(AVG(DATE_DIFF(o.order_delivered_customer_date,
o.order_purchase_timestamp, DAY)), 2) AS avg_delivery_time
        FROM BusinessCaseTargetSQL.customers c
        JOIN BusinessCaseTargetSQL.orders o
            ON c.customer_id = o.customer_id
        WHERE o.order_delivered_customer_date IS NOT NULL
        GROUP BY c.customer_state
        ORDER BY avg_delivery_time ASC
        LIMIT 5
    )
    ORDER BY avg_delivery_time ASC;

```

Insight : Delivery is fastest in SP, PR, MG, DF, and SC, while PA, AL, AM, AP, and RR experience the longest average delivery times.

V.D. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

Job information		Results	Visualization	J
Row	customer_state ▼	avg_delivery_fast...		
1	AC	19.76		
2	RO	19.13		
3	AP	18.73		
4	AM	18.61		
5	RR	16.41		

Query: `SELECT *`
`FROM (`
`SELECT`
`c.customer_state,`
`ROUND(AVG(DATE_DIFF(o.order_estimated_delivery_date,`
`o.order_delivered_customer_date, DAY)), 2) AS avg_delivery_faster_than_estimate`
`FROM BusinessCaseTargetSQL.customers c`
`JOIN BusinessCaseTargetSQL.orders o`
`ON c.customer_id = o.customer_id`
`WHERE o.order_delivered_customer_date IS NOT NULL`
`GROUP BY c.customer_state`
`ORDER BY avg_delivery_faster_than_estimate DESC`
`LIMIT 5`
`);`

Insight : Orders are delivered much faster than estimated in AC, RO, AP, AM, and RR, with these states outperforming expectations.

VI.A. Find the month on month no. of orders placed using different payment types.

Job information	Results	Visualization	JSON	Execution details	Exe
Row	order_year_month	payment_type		total_orders	
1	2016-09	credit_card		3	
2	2016-10	UPI		63	
3	2016-10	credit_card		253	
4	2016-10	debit_card		2	
5	2016-10	voucher		11	
6	2016-12	credit_card		1	
7	2017-01	UPI		197	
8	2017-01	credit_card		582	
9	2017-01	debit_card		9	
10	2017-01	voucher		33	
11	2017-02	UPI		398	

Query: **SELECT**

```
    FORMAT_TIMESTAMP('%Y-%m', o.order_purchase_timestamp) AS order_year_month,
    p.payment_type,
    COUNT(DISTINCT o.order_id) AS total_orders
FROM BusinessCaseTargetSQL.orders o
JOIN BusinessCaseTargetSQL.payments p
    ON o.order_id = p.order_id
GROUP BY order_year_month, p.payment_type
ORDER BY order_year_month, p.payment_type;
```

Insight : Credit cards are the most commonly used payment method across all months.

VI.B. Find the no. of orders placed on the basis of the payment installments that have been paid.z

Job information		Results	Visualizatio
Row	payment_installm...	total_orders ▼	
1	0	2	
2	1	49057	
3	2	12389	
4	3	10443	
5	4	7088	
6	5	5234	
7	6	3916	
8	7	1623	
9	8	4253	
10	9	644	
11	10	5315	
12	11	23	
--	--	---	

Query: `SELECT
 payment_installments,
 COUNT(DISTINCT order_id) AS total_orders
FROM BusinessCaseTargetSQL.payments
WHERE payment_value > 0
GROUP BY payment_installments
ORDER BY payment_installments;`

Insight : Most Brazilian customers prefer 1–3 installment payments, with very few opting for long-term installment plans..